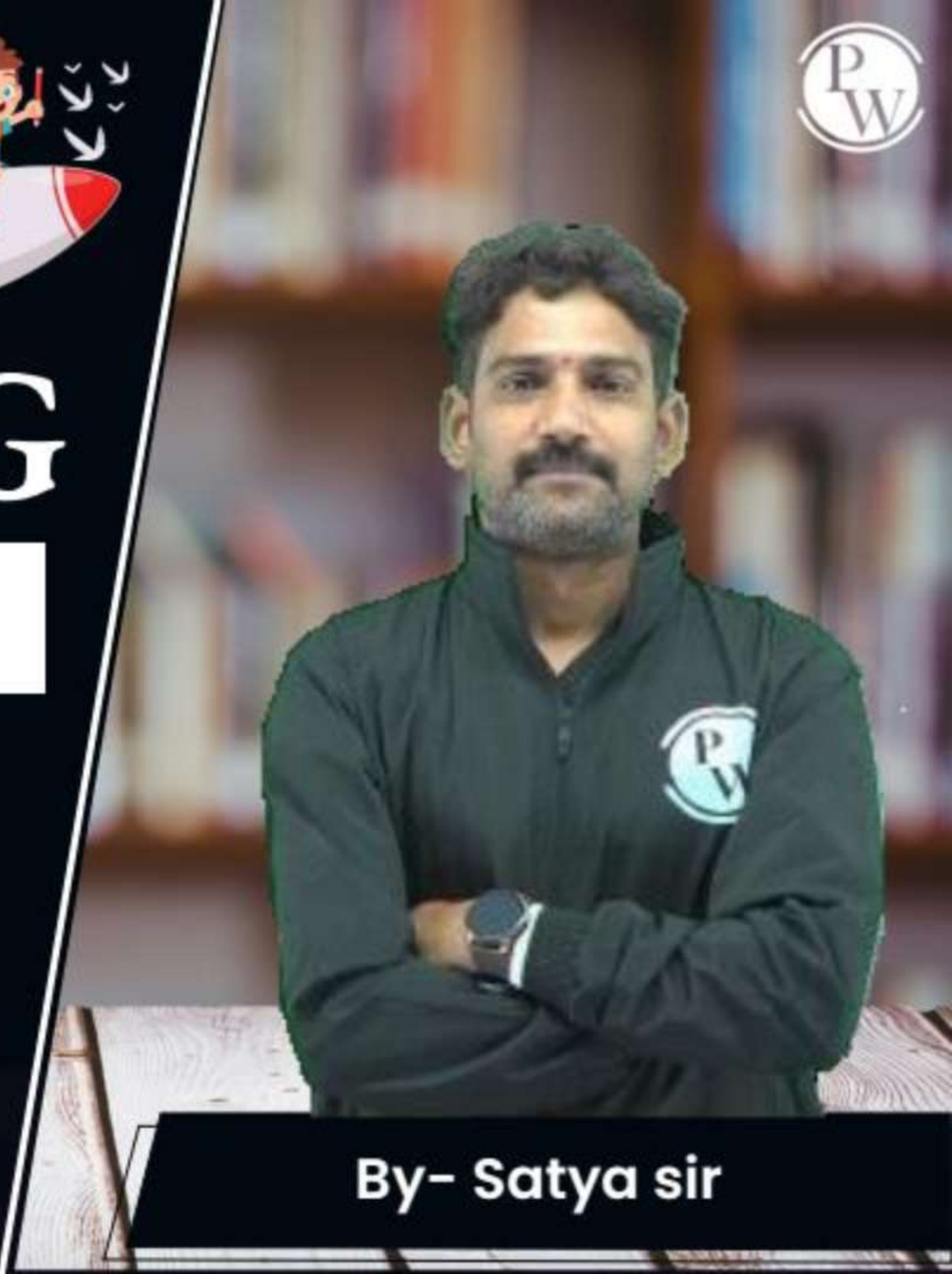
CS & IT ENGINEERING

'C' Programming

Pointers & Arrays

Lecture No.- 04



By- Satya sir

Recap of Previous Lecture

Strings

- Character array or Character Pointer
- Strlen( )
- Sizeof( )
- Declaration and Initialization

Topics to be Covered

- String handling functions
- 2D-array
  - Declaration
  - Initialization
  - RMO, CMO
  - Address of an Element
  - Programs*

Topic : String Handling in 'C'

No.	Function	Description
1)	strlen(string_name)	returns the length of string name.
2)	strcpy(destination, source)	copies the contents of source string to destination string.
3)	strcat(first_string, second_string)	concats or joins first string with second string. The result of the string is stored in first string.
4)	strcmp(first_string, second_string)	compares the first string with second string. If both strings are same, it returns 0.
5)	strrev(string)	returns reverse string.
6)	strlwr(string)	returns string characters in lowercase.
7)	strupr(string)	returns string characters in uppercase.
8)	strstr(str1, str2)	It returns a pointer to the first occurrence of the given substring str2 within the given string str1

Topic : String Handling in 'C'

No.	Function	Description
9)	strncmp()	It compares two strings only to n characters.
10)	strncat()	It concatenates n characters of one string to another string.
11)	strncpy()	It copies the first n characters of one string into another.
12)	strchr()	It finds out the first occurrence of a given character in a string.
13)	strrchr()	It finds out the last occurrence of a given character in a string.
14)	strnstr()	It finds out the first occurrence of a given string in a string where the search is limited to n characters.
15)	strcasecmp()	It compares two strings without sensitivity to the case.
16)	strncasecmp()	It compares n characters of one string to another without sensitivity to the case.

Topic : String Handling in 'C'

```
char S1[ ]="GATE", S2[ ]="EXAM";
char S3[10], S4[ ]="Gate", *S5;
```

```
Strcpy(S1,S2);      // S1 = "EXAM"
S3 = Strcat(S1,S2); // S1 = "EXAMEXAM"
```

Error
Array assignment

```
S3 = Strrev(S2); // S3 = "MAXE"
S5 = Strupr(S4); // S5 = "GATE"
S1 = Strlwr(S2); // S1 = "exam"
```

Syntax:

Strcmp(S1, S2)

- returns 0 — S1 == S2
- returns +ve — S1 > S2
- returns −ve — S1 < S2

- Lexicographical Comparison

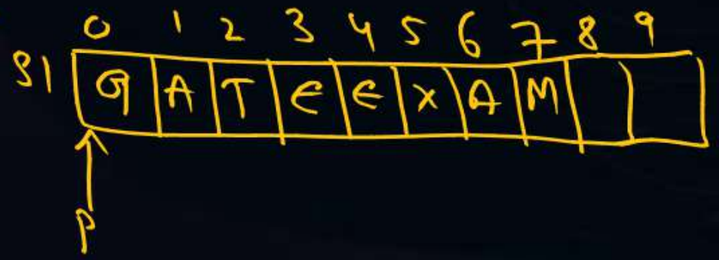
when characters in both S1 and S2 at i^{th} Position are same Then only $(i+1)^{th}$ characters are compared. otherwise [i^{th} char of S1 − i^{th} char of S2]

String as character Pointer

```
char *Name;
```

```
char *P = "GATE";
Printf("%s", P); //GATE
Printf("%c", P); //Error, P Points to whole String, not single character.
```

```
char S1[10], *P;
P = S1;
for(i=0; i<10; i++)
    Scanf("%c", S1[i]); (OR) Scanf("%c", P+i);
```



Topic : 2-D Array

2-D Arrays

- It is also known as matrix

Declaration : datatype arrayname [1st dimension size] [2nd dimension size]

datatype arrayname [Number of rows] [Number of columns];

Ex:
```
int x[3][4]; // Total elements 3x4=12
char y[2][4];
float z[4][5];
```

Initialization : **Syntax:** datatype Name [rows] [cols] = { Value(s) };

```
int x[2][3]={10,20,30,40,50,60};
int x[2][3]={10,20,30,40};
int x[2][3]={10};
```

Topic : 2-D Array

Syntax 2 :

datatype arrayname [rows] [cols] = { { row 0 values }, { row1 values } .... { Last row values } }

int x[2][3] = { {10,20,30}, {40,50,60} };

---

int x[2][3] = {10, 20, 40, 50}

x	0	1	2
0	10	20	40
1	50	0	0

(x: rows, cols)

int x[2][3] = { {10,20}, {40,50} }

x	0	1	2
0	10	20	0
1	40	50	0

Representation Model

Topic : 2-D Array

- In memory, all array elements stored at Contiguous Locations. [Consecutive / continuous locations].

- 2-D array elements are stored in either of 2 ways:

1) Row-Major order (RMO) : Row-wise : C, C++, Java, Python

2) Column-Major order (CMO) : Col-wise : Fortran, Scilab, MATLAB

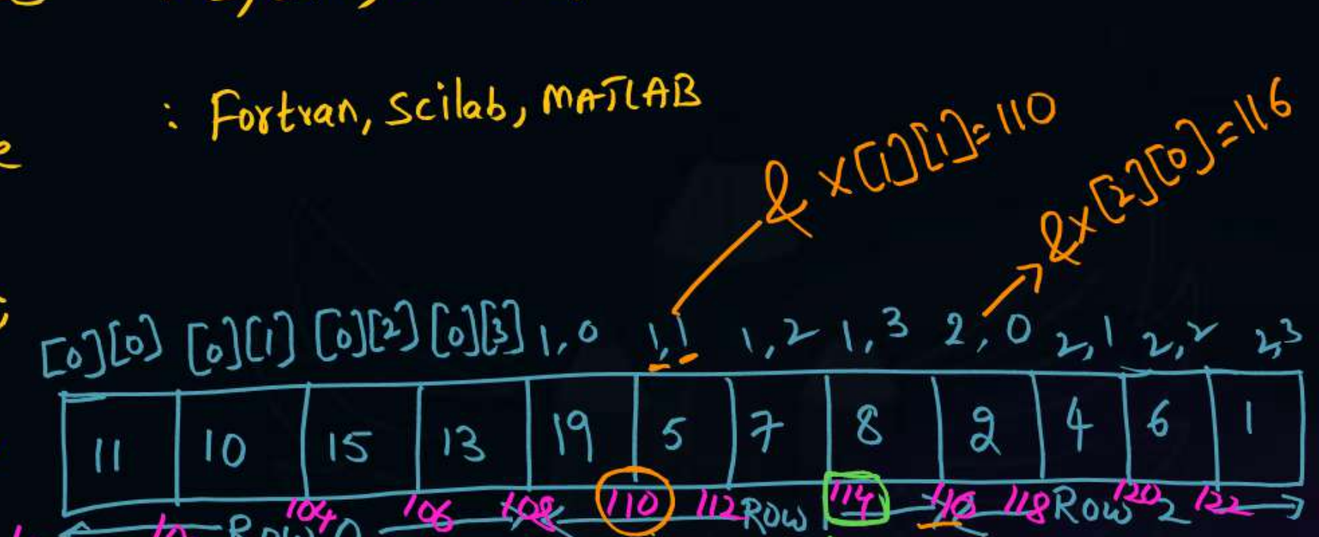
Ex: Let 2 Bytes Per integer, Base address = 100

```
int x[3][4] = {11, 10, 15, 13, 19, 5, 7, 8, 2, 4, 6, 1};
```

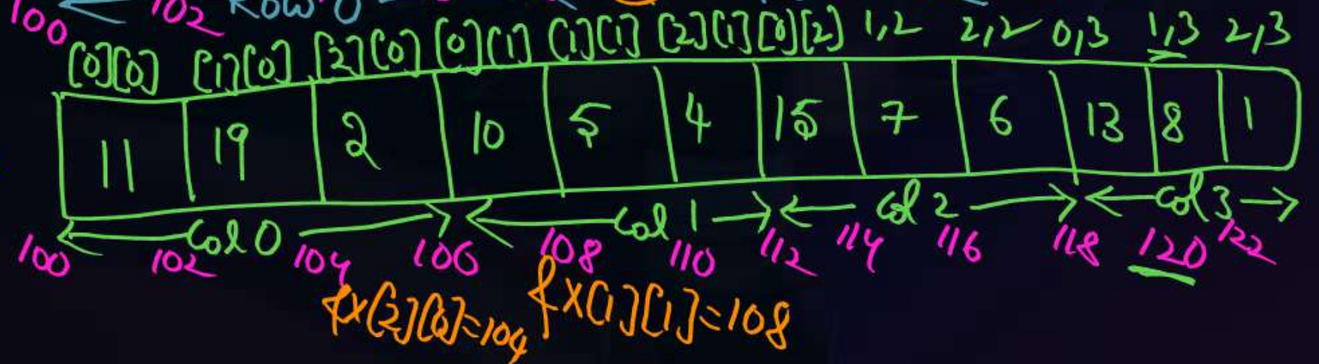
Representation

x	0	1	2	3
0	11	10	15	13
1	19	5	7	8
2	2	4	6	1

RMO:



CMO:



Topic : 2-D Array

Address of an Element

1-D Array : Let x[ ] be 1-D array

B = Base address

n = Size of Each Element

i = index

$$\&x[i] = B + i * n$$

2-D Array : Let x[R][C] be 2-D array

B = Base address

R = No. of rows

C = No. of cols

N = Size of Each Element

i = row index

j = col index

$$\text{RMO: } \&x[i][j] = B + [i * C + j] * N$$

$$\text{CMO: } \&x[i][j] = B + [j * R + i] * N$$

Ex: R = 3 N = 2
C = 4 B = 100

$\&x[i][j]$

$\&x[1][3]$

RMO : $100 + (1*4 + 3)*2 = 100 + 14 = 114$

CMO : $100 + (3*3 + 1)*2 = 100 + 20 = 120$

Topic : 2-D Array

```
// I/O 2-D array elements
int main( )
{
   int x[10][10], r, c, i, j;
   Printf("Enter No. of rows and cols");
   scanf("%d %d", &r, &c);
/* For input 2-D array */
   for( i=0 ; i<r; i++)
   { for( j=0; j<c; j++)
      { scanf("%d", &x[i][j]);
      }
   }
```

```
/* To Print 2-D array Element Values */
   for(i=0; i<r; i++)
   { for( j=0;j<c; j++)
      { Printf("%d", x[i][j]);
      }
      Printf("\n");
   }
  return 0;
}
```

2 mins Summary

- String handling functions
- 2-D array
  - Declaration
  - Initialization
  - Address of Element
    - RMO
    - CMO
  - I/O 2-D array

To be contd...

THANK - YOU